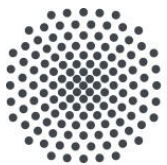


Collaborative Model-Driven Quantum Software Engineering: A Hands-On Tutorial with Qonstruct



University of Stuttgart

Lavinia Stiliadou

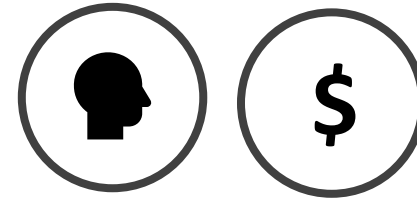
stiliadou@iaas.uni-stuttgart.de

Institute for Architecture of Application Systems

Motivation

- Classical Bit

- like a coin, either 0 (head) or 1 (tail)



- Qubit

- can be both tails and heads at the same time



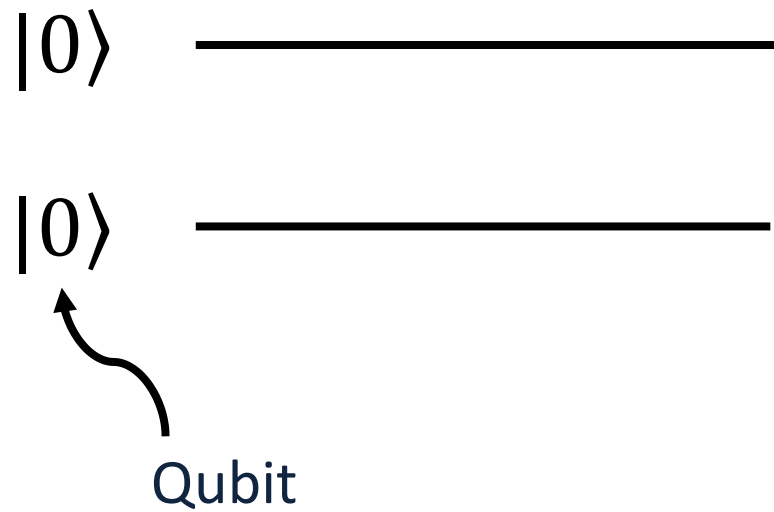
- Superposition

- many states at the same time

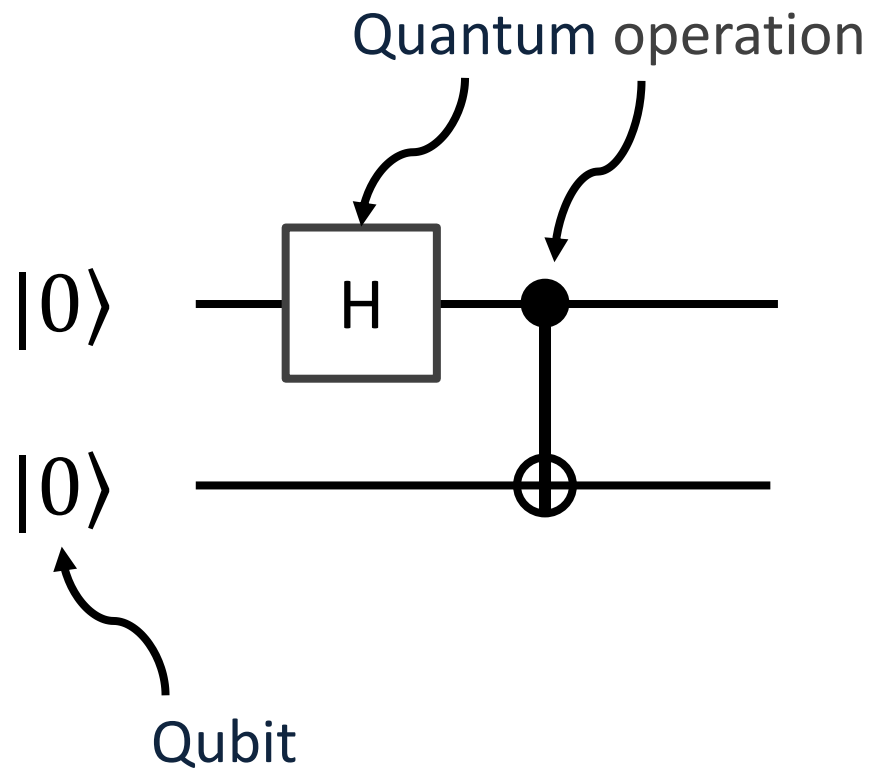
- Measurement

- forces the qubits to collapse into a definite classical value

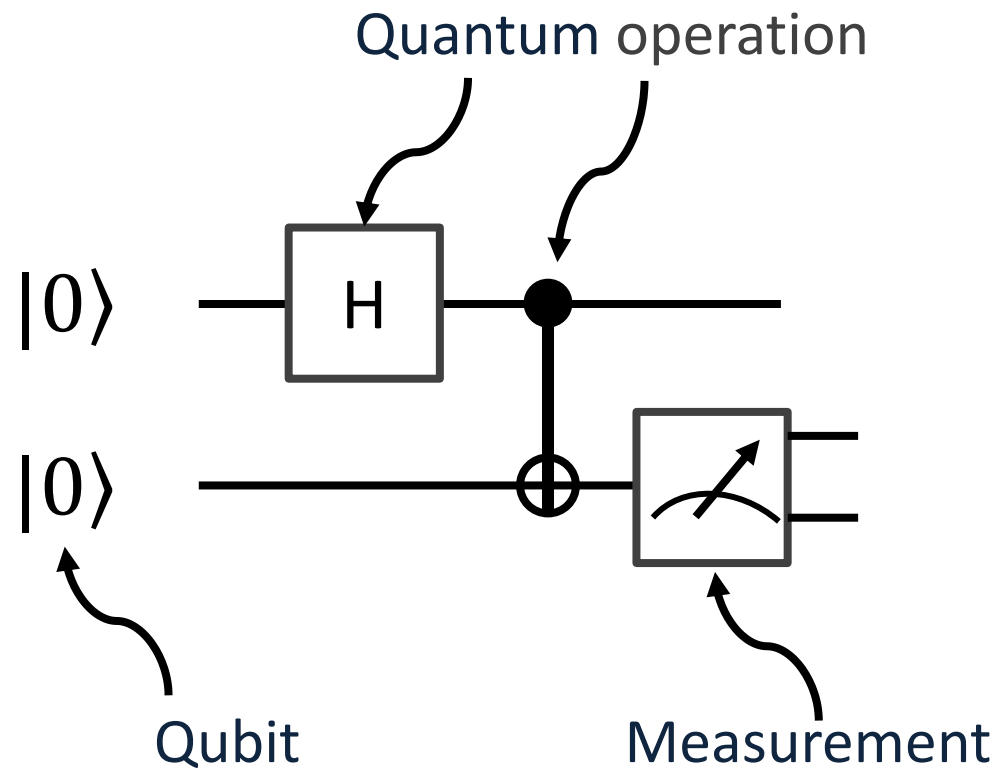
Motivation



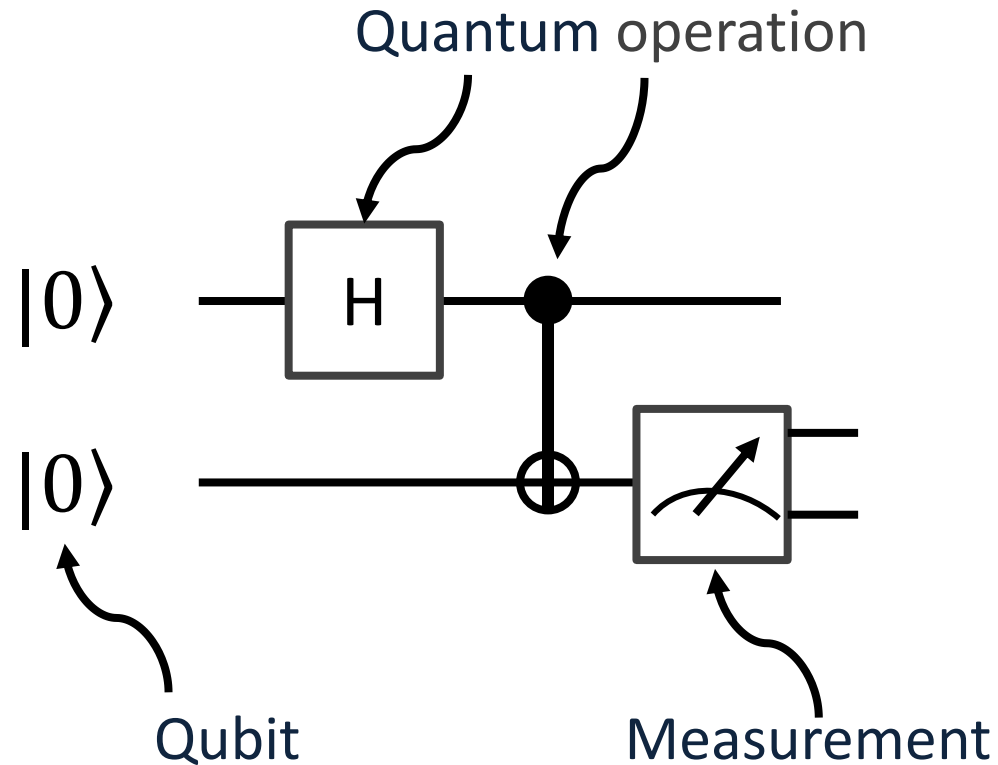
Motivation



Motivation



Motivation



```
// Imports
```

```
qc = QuantumCircuit(2, 1)
```

```
qc.h(0)
```

```
qc.cx(0, 1)
```

```
qc.measure(0, 0)
```

Motivation

- Existing quantum low-code platforms:
 - Often closed-source
 - Vendor-locked
 - Circuit-level only

Research Question

- To make quantum computing accessible:
 - Users need visual abstractions instead of low-level coding
 - Hybrid quantum-classical operations must be represented intuitively
 - Different expertise levels must be supported
 - Models should remain reusable and extensible

Research Question:

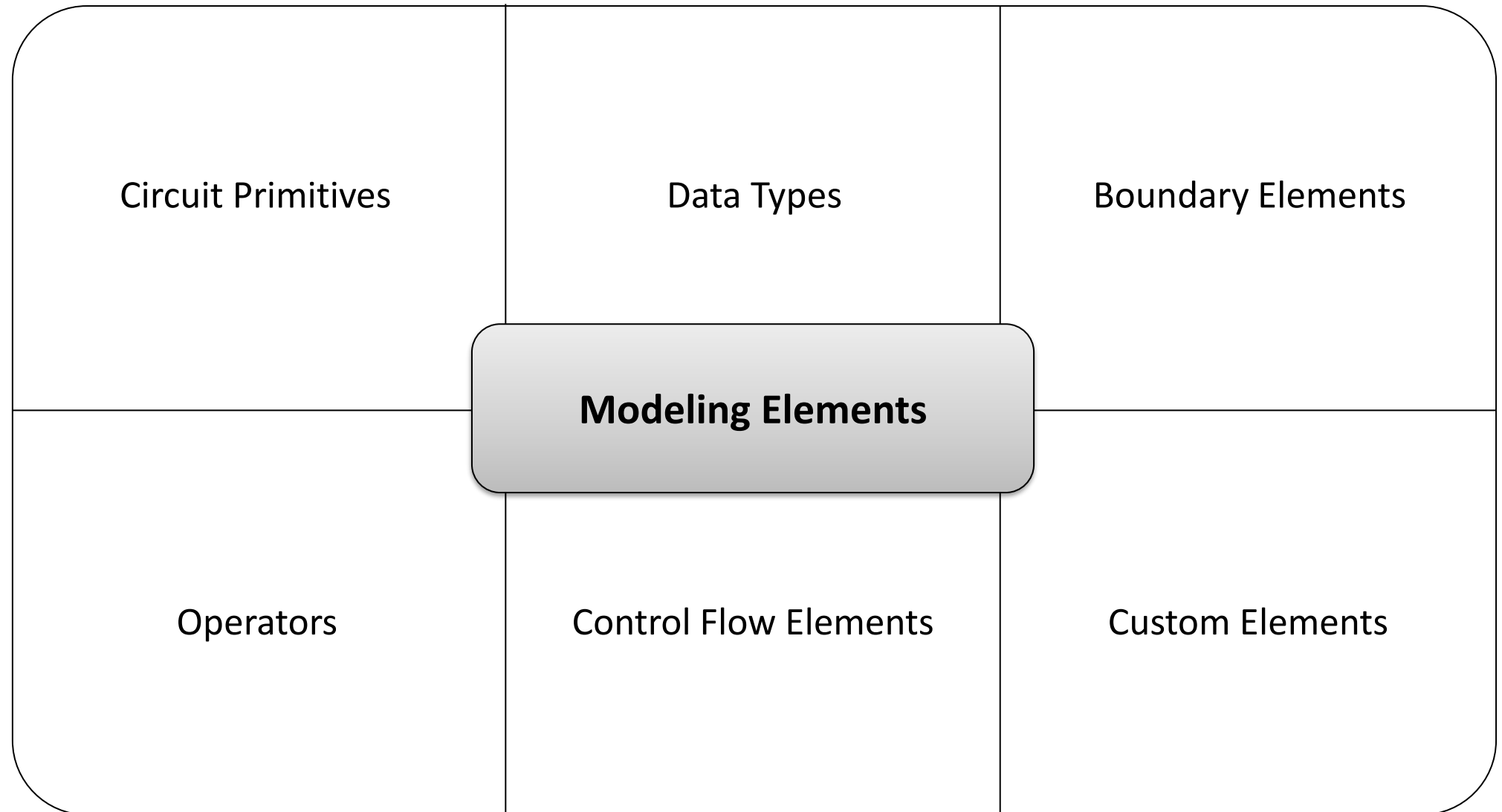
Which **abstractions** and **modeling elements** are required to support modeling of quantum algorithms in a **low-code platform**?

Modeling Language

- Icon
 - Basis Encoding Pattern Icon
- Type
 - State Preparation
- Properties
 - Encoding Type
 - Bound
 - Implementation (optional)
- Inputs
- Value
 - Value
- Outputs
 - Quantum Register

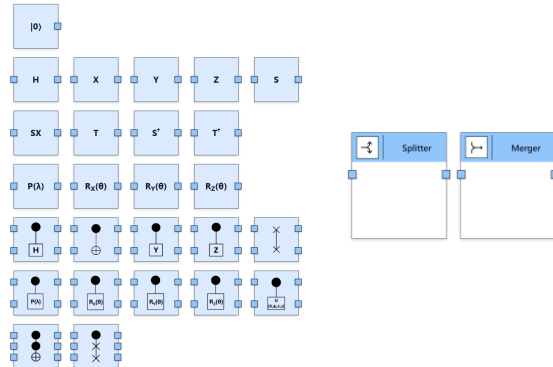
The diagram illustrates the 'Encode Value' block, which is a component in a modeling language. It features a blue header bar with the title 'Encode Value' and a small icon of a quantum circuit on the left. Below the header, there are two main sections. The first section, labeled 'Encoding Type:', contains a dropdown menu currently set to 'Basis Encoding'. Below this, the 'Bound:' section has a text input field with the placeholder 'Enter bound'. A yellow callout bubble points to the block, indicating 'Value type: any'. The second section, labeled 'Output: type: quantum register', is located in the bottom right corner and contains two input fields: 'Identifier' (a solid blue border) and 'Size' (a dashed blue border). A small blue square is positioned to the right of the 'Identifier' field.

Modeling Language



Modeling Language

Circuit Primitives



Data Types

Boundary Elements

Modeling Elements

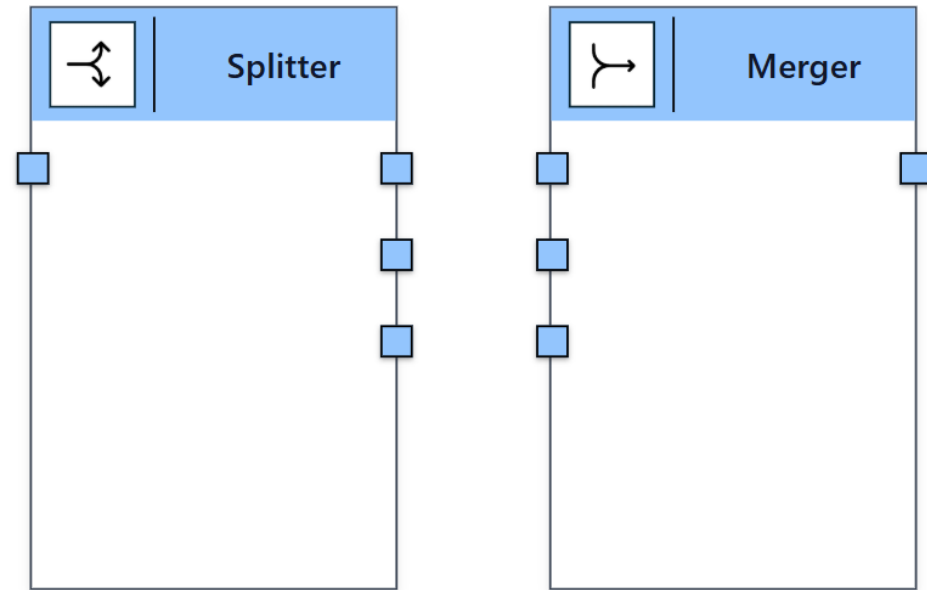
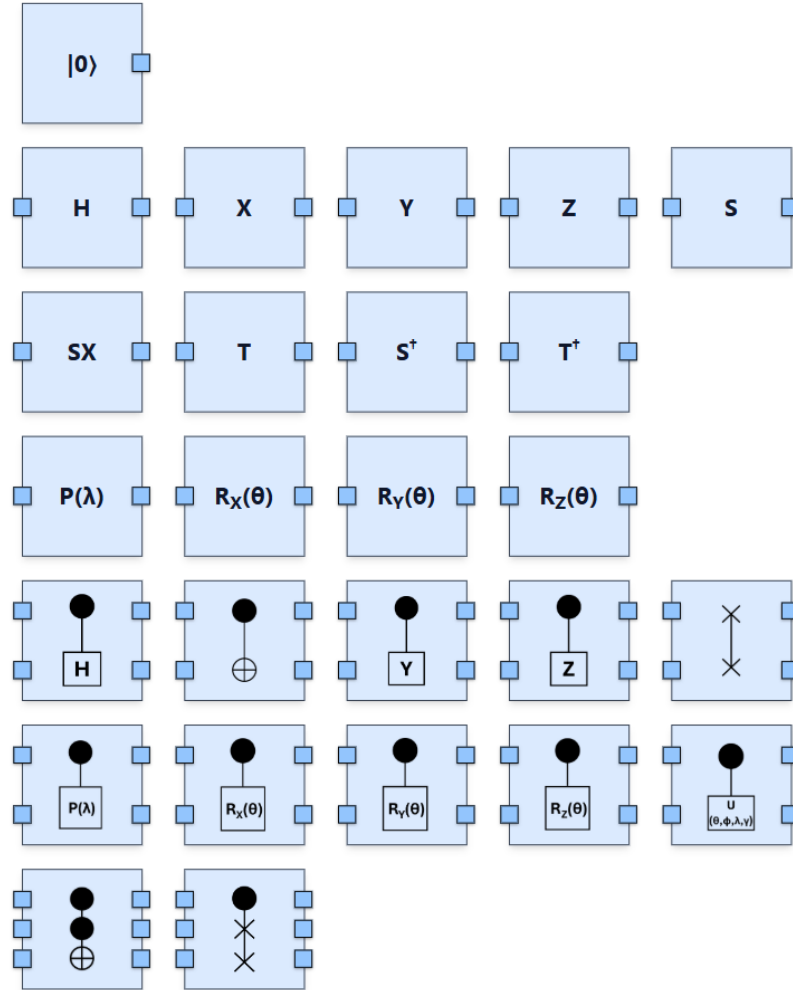
Operators

Control Flow Elements

Custom Elements

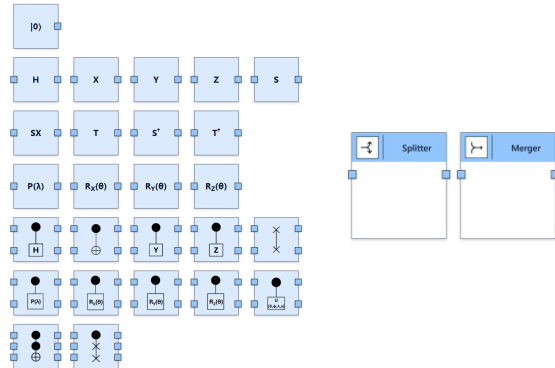
Modeling Language

■ Circuit Primitives:



Modeling Language

Circuit Primitives



Data Types

Boundary Elements

Modeling Elements

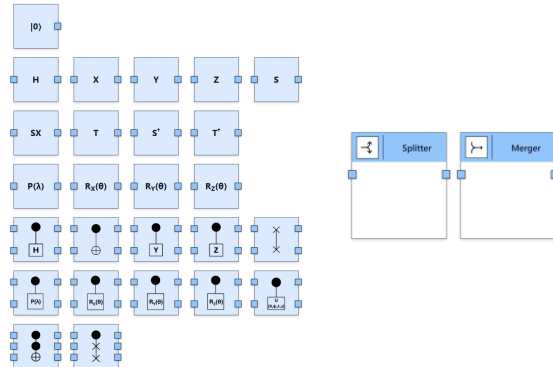
Operators

Control Flow Elements

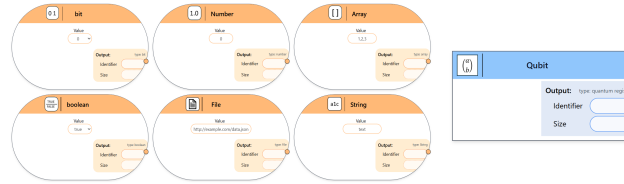
Custom Elements

Modeling Language

Circuit Primitives



Data Types



Boundary Elements

Modeling Elements

Operators

Control Flow Elements

Custom Elements

Modeling Language

■ Data Types:

0 1

bit

Value
0 ▾

Output: type: bit
Identifier
Size

1.0

Number

Value
0

Output: type: number
Identifier
Size

[]

Array

Value
1,2,3

Output: type: array
Identifier
Size

TRUE
FALSE

boolean

Value
true ▾

Output: type: boolean
Identifier
Size



File

Value
<http://example.com/data.json>

Output: type: File
Identifier
Size

a1c

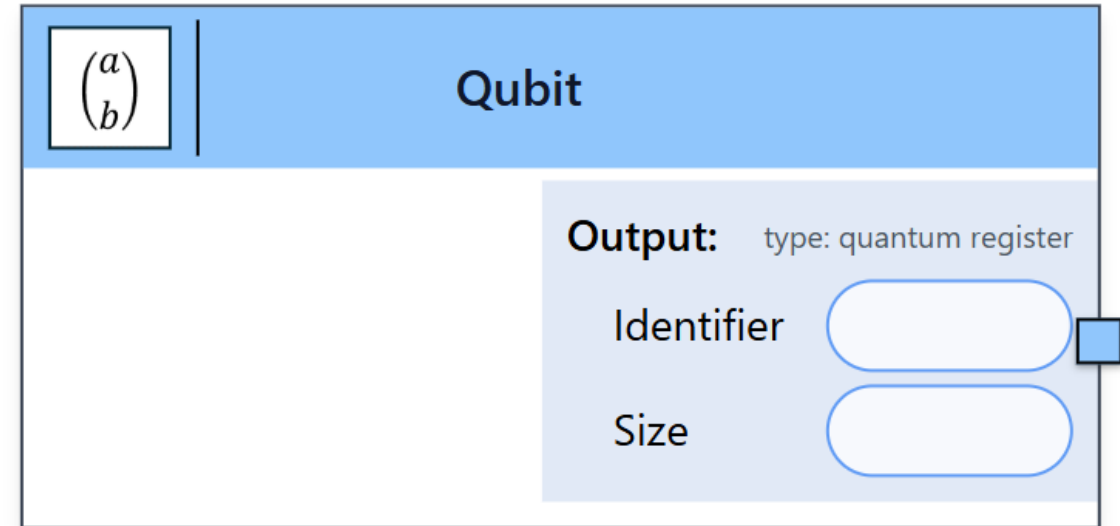
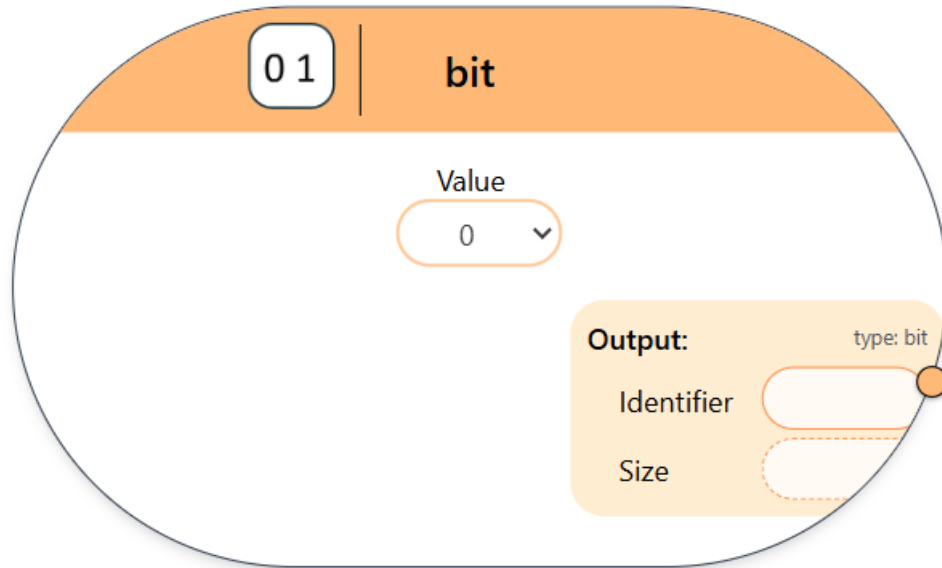
String

Value
text

Output: type: String
Identifier
Size

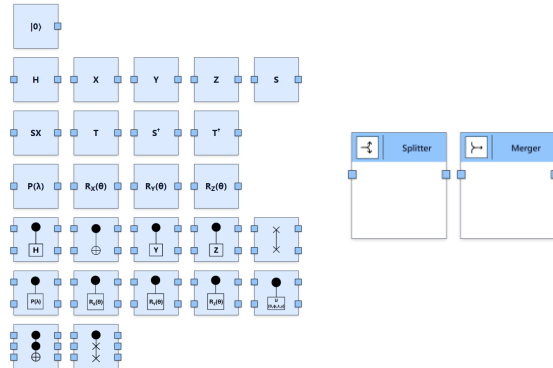
Modeling Language

- Data Types:

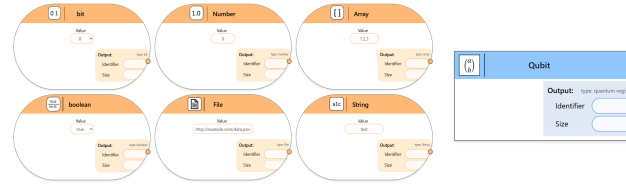


Modeling Language

Circuit Primitives



Data Types



Boundary Elements

Modeling Elements

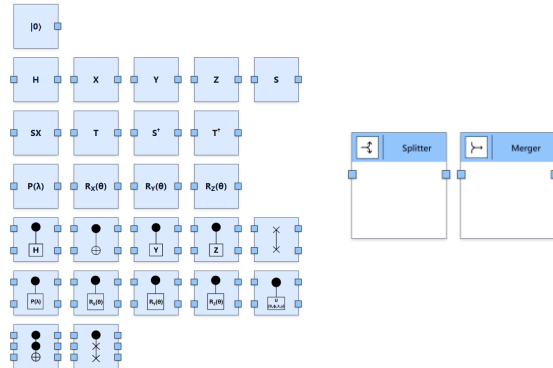
Operators

Control Flow Elements

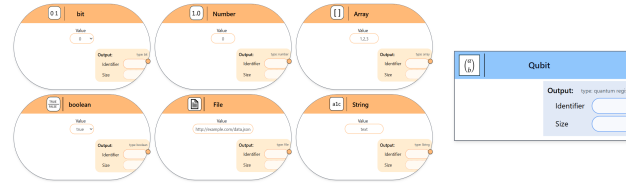
Custom Elements

Modeling Language

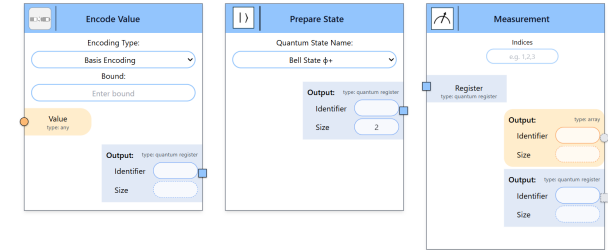
Circuit Primitives



Data Types



Boundary Elements



Modeling Elements

Operators

Control Flow Elements

Custom Elements

Modeling Language

■ Boundary Elements:

 **Encode Value**

Encoding Type:

Basis Encoding

Bound:

Enter bound

Value

type: any

Output: type: quantum register

Identifier

Size

 **Prepare State**

Quantum State Name:

Bell State $\phi+$

Output: type: quantum register

Identifier

Size

2

 **Measurement**

Indices

e.g. 1,2,3

Register

type: quantum register

Output: type: array

Identifier

Size

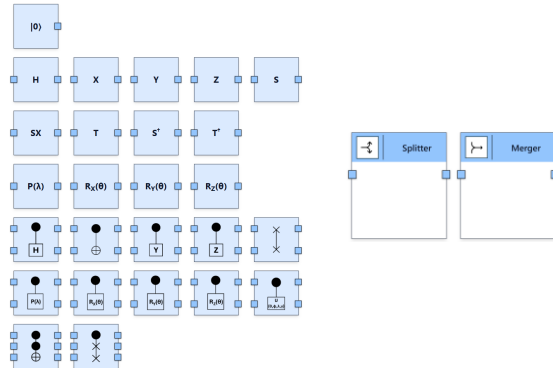
Output: type: quantum register

Identifier

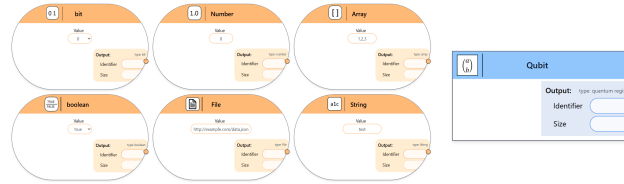
Size

Modeling Language

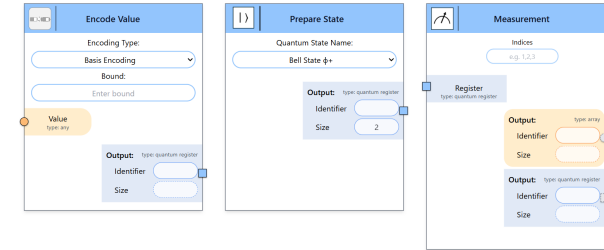
Circuit Primitives



Data Types



Boundary Elements



Modeling Elements

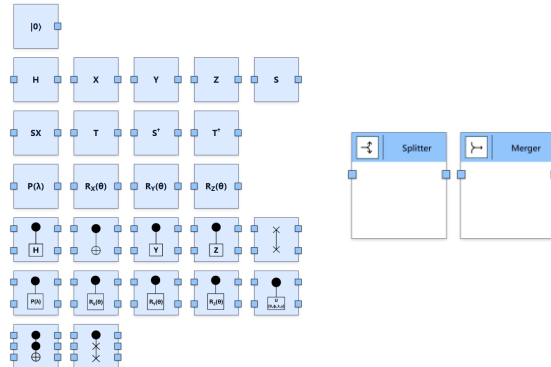
Operators

Control Flow Elements

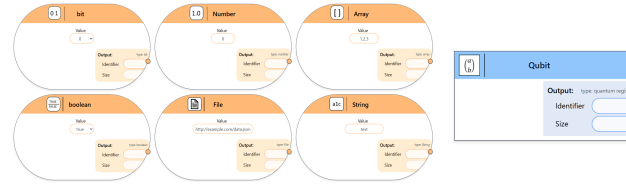
Custom Elements

Modeling Language

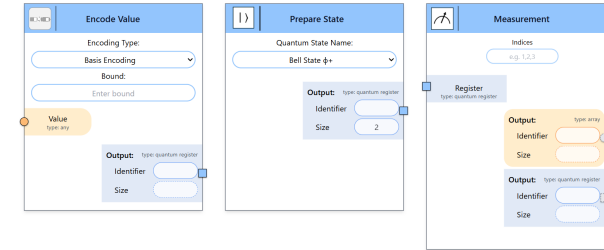
Circuit Primitives



Data Types

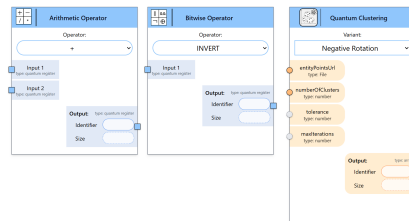


Boundary Elements



Modeling Elements

Operators



Control Flow Elements

Custom Elements

■ Operatoren:

+

-

/

*

Arithmetic Operator

Operator:

+

Input 1
type: quantum register

Input 2
type: quantum register

Output: type: quantum register

Identifier

Size

||

&&

¬

⊕

Bitwise Operator

Operator:

INVERT

Input 1
type: quantum register

Output: type: quantum register

Identifier

Size

Quantum Clustering

Variant:

Negative Rotation

entityPointsUrl
type: File

numberOfClusters
type: number

tolerance
type: number

maxIterations
type: number

Output: type: array

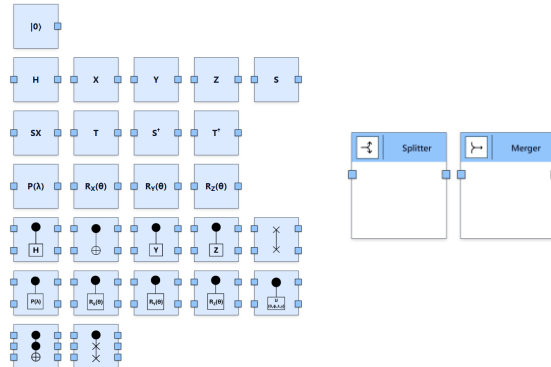
Identifier

Size

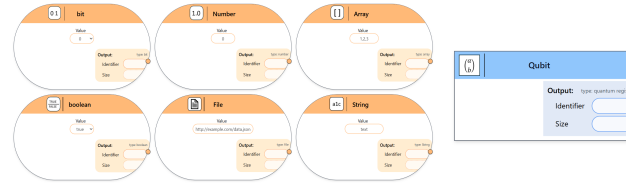
...

Modeling Language

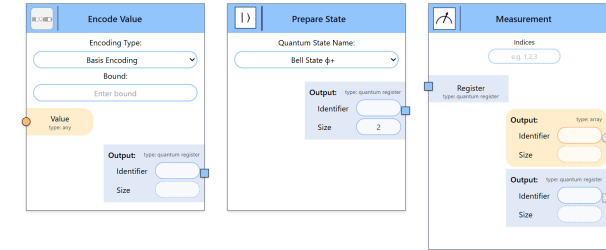
Circuit Primitives



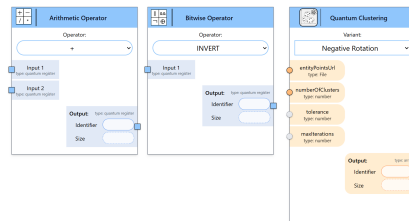
Data Types



Boundary Elements



Modeling Elements



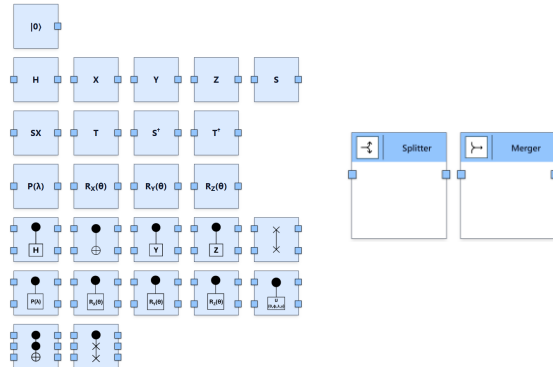
Operators

Control Flow Elements

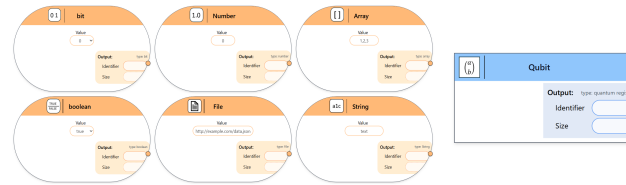
Custom Elements

Modeling Language

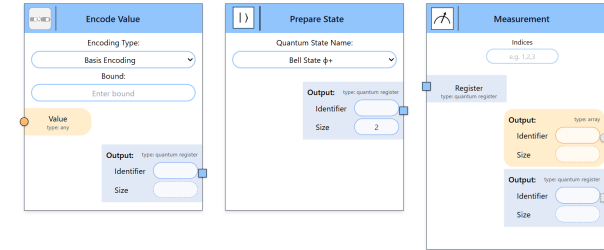
Circuit Primitives



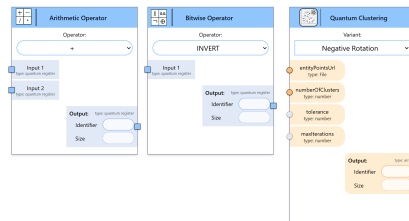
Data Types



Boundary Elements

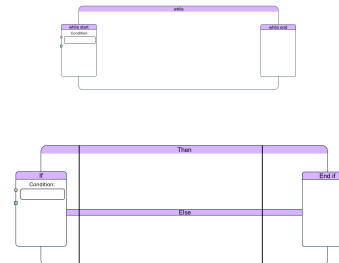


Modeling Elements



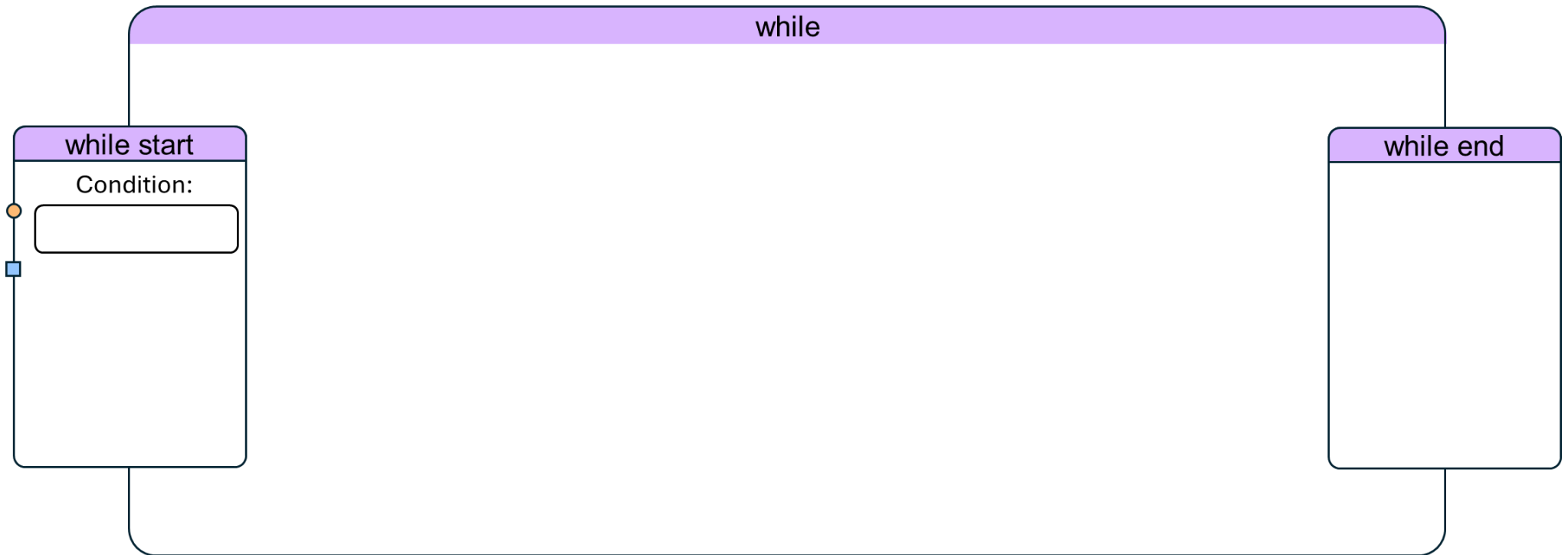
Operators

Control Flow Elements

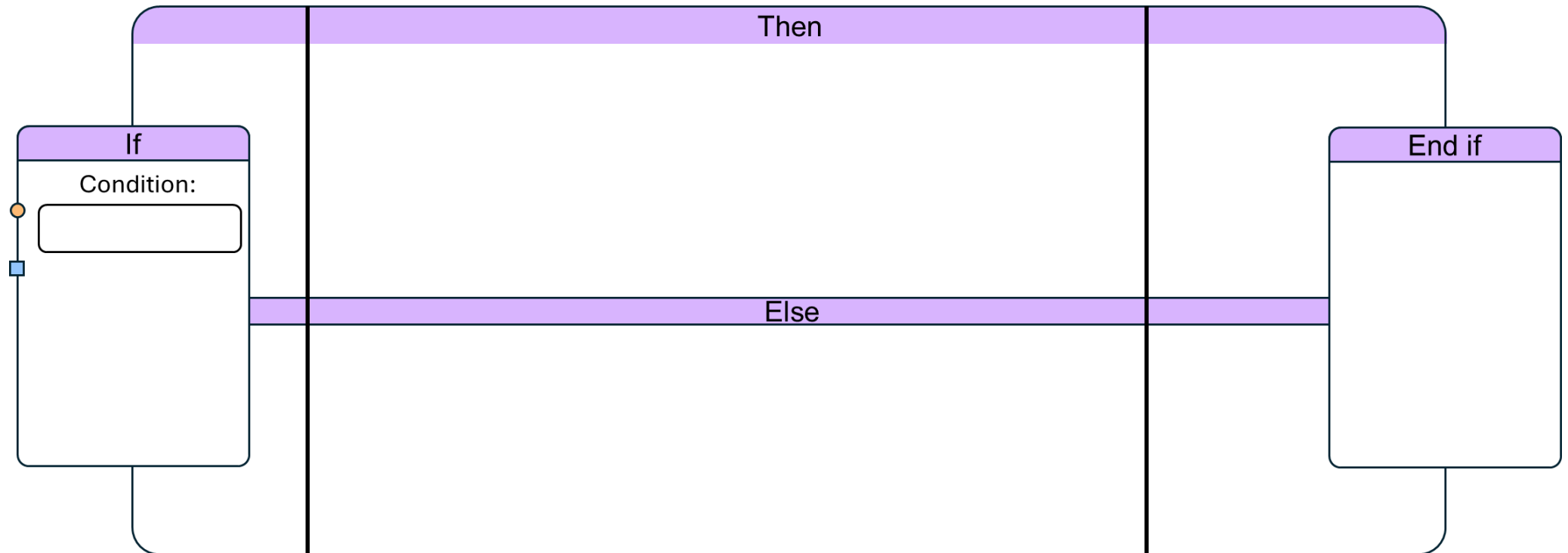


Custom Elements

- Control Flow Elements:

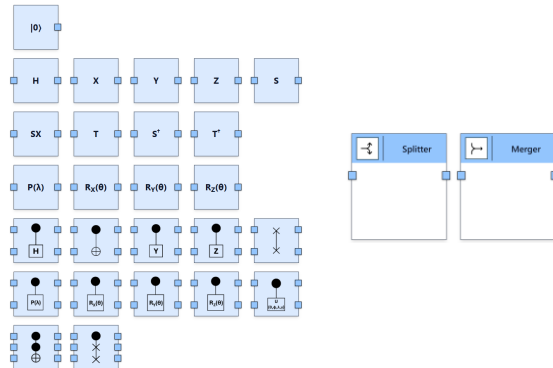


- Control Flow Elements:

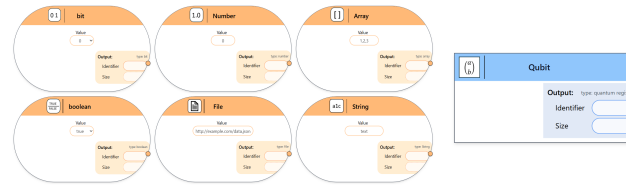


Modeling Language

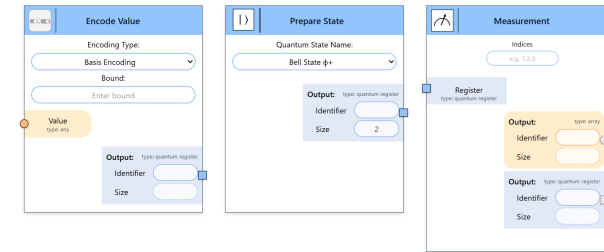
Circuit Primitives



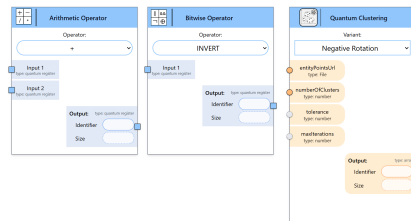
Data Types



Boundary Elements

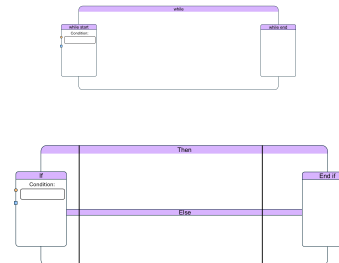


Modeling Elements

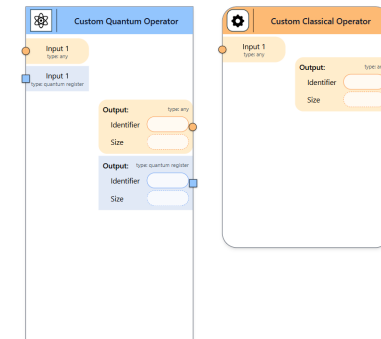


Operators

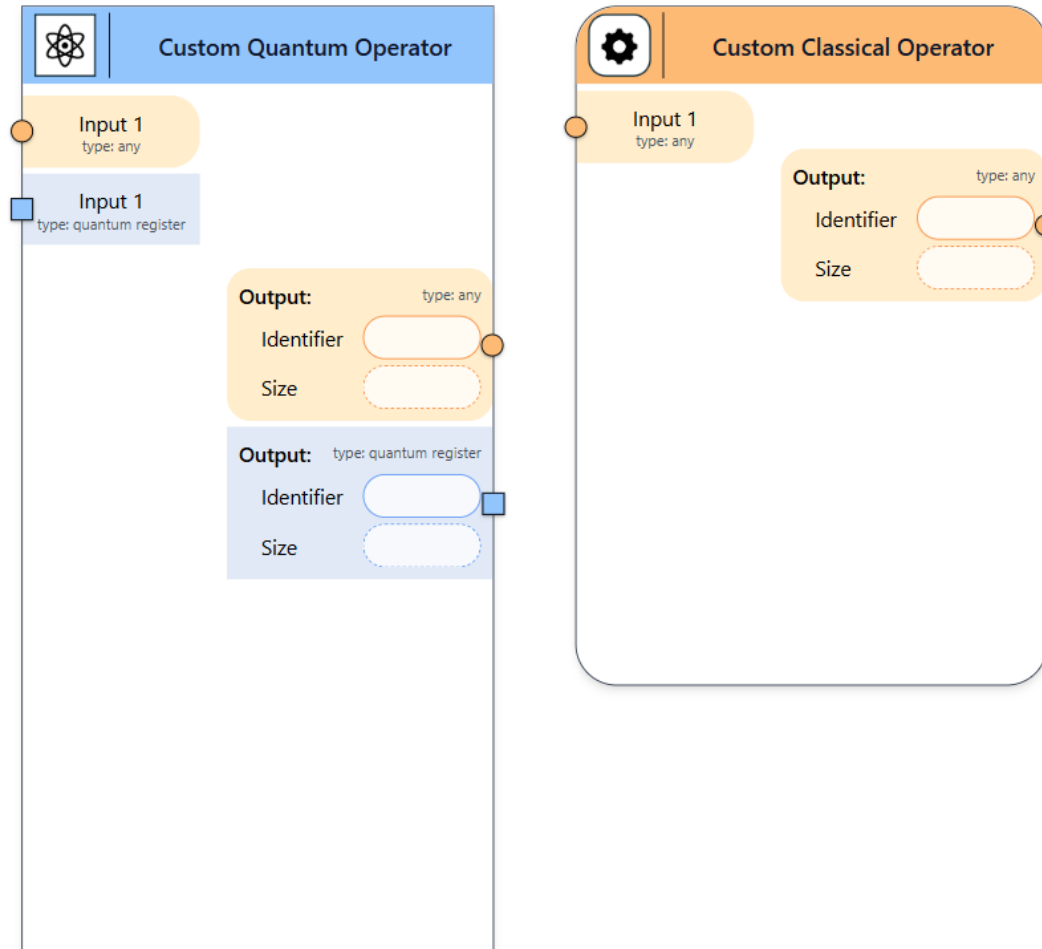
Control Flow Elements



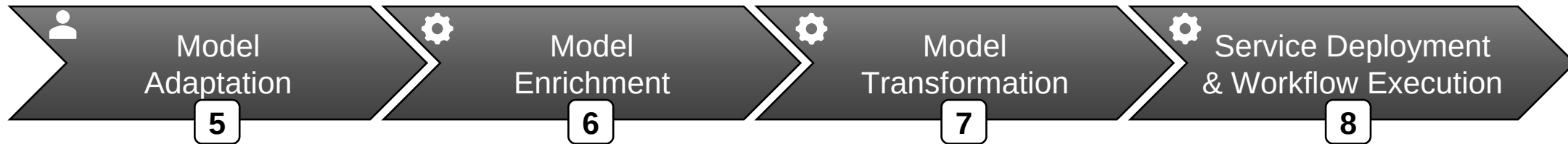
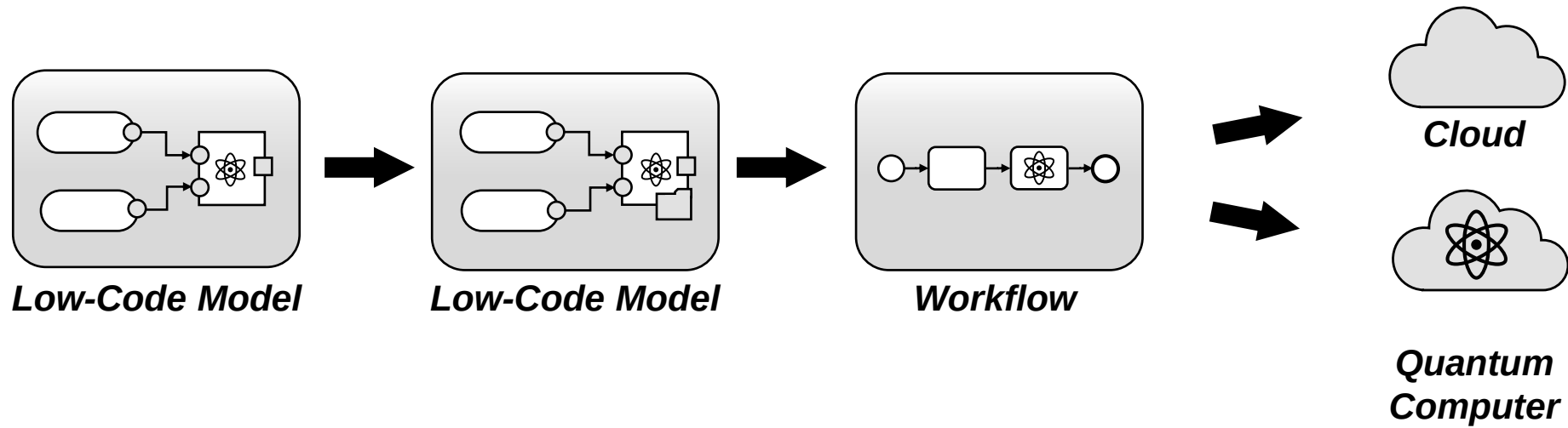
Custom Elements



■ Custom Elements:



Modeling and Execution of Quantum Algorithms



Legend:



Manual
Phase



Automated
Phase



Algorithm



Quantum



Implementation

Thank you for your attention!